你的 Harness 工作流真的在进步吗？我们用一场考试撕掉了遮羞布

原创 腾讯程序员 腾讯技术工程 2026年6月18日 03:20 内蒙古 279人



作者：chaseren

> Harness Eval：一套面向 Harness 工作流的、轻量的、可回归的闭环评测系统。

一、一个让人不安的事实：你的Harness工作流可能一直在“裸奔”

你有没有这样的经历？

团队花了两周精心调教一套 Harness 研发工作流——写了十几条 Rules、打磨了 Skill——上线之后，评价体系是这样的：

"感觉这版稳了不少。"

"昨天那版好像更聪明？"

"我这边挺好用，你那边咋不行？"

这就是我们半年前面对的现实。整个团队在用"主观 vibes"驱动一个复杂系统的演进。

说白了，我们连最基本的问题都回答不了：

- **改了一版 rule/skill，到底是进步还是退步？** 典型的"主观 vibes"驱动开发无法量化这个事情。
- **A 说好用，B 说难用——到底谁对？** 工作流问题、模型问题、还是 Prompt 本身有歧义？说不清。
- **工作流回归无从谈起。** 修一个 bad case，可能顺手破坏了三个 good case，但你不知道。
- **我们的 Harness 工作流和竞品比，到底差在哪？** 没有数据来衡量和佐证，都是各吹各的好。

这不是一个"锦上添花"的问题。这是一个**系统正在失控而你不自知**的问题。

二、核心哲学:为什么Harness工作流比传统软件更需要“考试”

2.1 不可量化的东西，不可优化

这是整套系统的第一性原理。

传统软件是确定性的：同样的输入，同样的输出。你写一个`asserEqual`

它的结果是二值的、稳定的、可复现的。

Harness 工作流完全不同。它本质上是一种**"规则驱动的概率程序"**——输出由 `Prompt + Rules + Skills + Model` 共同决定，任何一项的微调都可能引起蝴蝶效应般的行为变化。今天跑一遍是 A 结果，明天跑一遍可能是 B 结果。更要命的是，你甚至无法简单地判定"A 比 B 好"还是"B 比 A 好"，因为"好"本身就缺乏定义。

这种系统如果没有评测体系兜底，你面对的不是"缓慢退化"——而是**薛定谔式的退化**：你永远不知道它什么时候变差了。

2.2 从“测试”到“考试”的思维跃迁

软件工程界用了几十年总结出"测试驱动开发"。但是，传统的单元测试、集成测试那套东西搬到 Harness 上是不够的。

为什么？因为 Harness 工作流的"正确"不是一个 boolean——它是一个**光谱**。一个 Harness 工作流完成任务的方式可以有无数种，有些优雅有些笨拙，有些主动有些被动，有些严谨有些草率。你不能简单地 `assert output == expected`

所以我们需要的不是"测试"，而是"考试"：

- **测试**验证的是"对不对"——二值判定。
- **考试**评估的是"好不好"——多维度打分 + 证据 + 改进建议。

> 这个思维跃迁是整套系统的哲学基石：**我们不是在写测试用例，我们是在出考卷、组织考试、培养阅卷官。**

2.3 三个不可妥协的原则

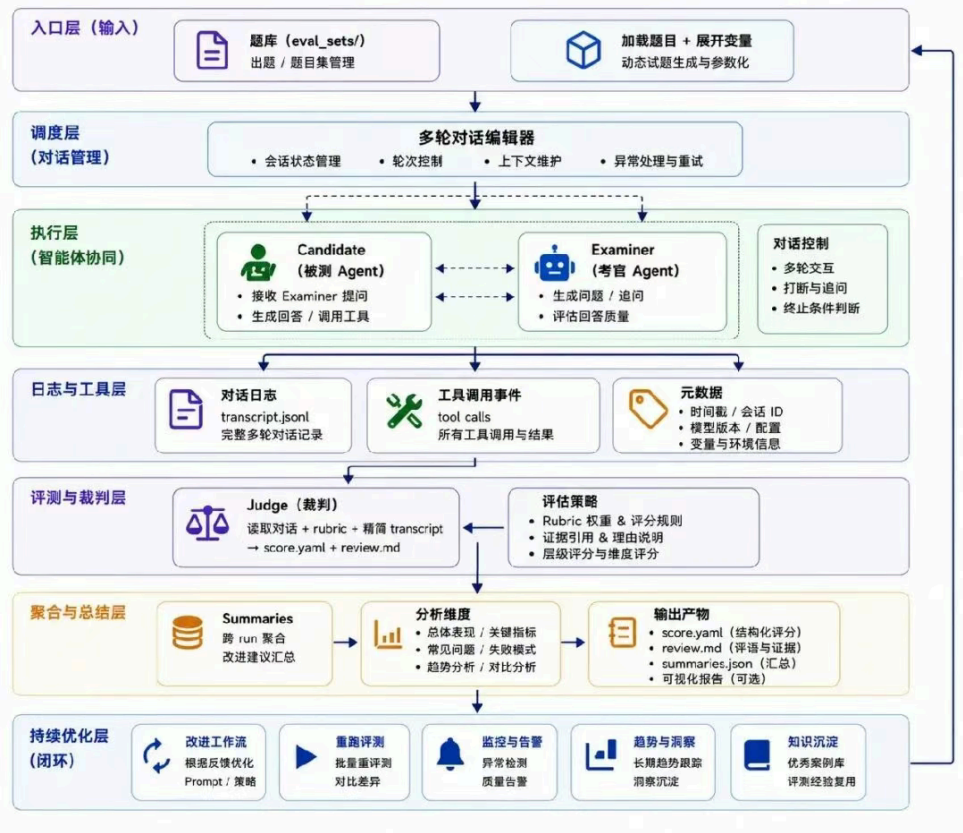
在设计整套系统之前，我们先锚定了三个原则，后来所有的设计决策都围绕它们展开：

- **可重复 > 精确。** 一道题跑多次，结果的分布比单次的分数更有价值。我们追求"统计显著的趋势"，不是"单次满分"。
- **可归因 > 高分。** 一道题失败了，系统必须能告诉你"为什么失败"——是工作流规则有漏洞？是题目出得有歧义？还是模型能力本身不够？分数不带归因，就是垃圾数据。
- **闭环 > 单向。** 评测的终点不是"出一份成绩单"，而是"驱动下一次改进"。如果跑完评测只能得到一个数字，却不知道下一步该改什么，那这个数字毫无意义。

三、系统全景：从一道题到一份成绩单

一张图回答"整个系统长什么样"。

整套系统只做三件事——**出题、答题、改卷**——但把它们串成了一个可以无人值守跑的闭环：



闭环的关键是把"改进"也纳入了链路：每次裁判输出的不只是分数，还会按 [工作流] / [题目] / [模型能力] 三个维度给出**可操作的改进建议**，直接喂回到下一轮工作流迭代里。

四、系统设计：出题→答题→改卷的闭环

4.1出题：如何设计一份AI考卷

一道题 = 题面 + 阅卷标准 + 环境前提 + 元信息，缺一不可。

每道题在题库里就是一个独立目录，由 4 个文件构成（外加可选的 fixtures），各司其职：

文件	角色	类比
meta.yaml	题目身份（id、版本、类别、难度、考察目的）	试卷头
task.md	题面（给考生看的题目原话 + 给考官看的剧本）	题干
rubric.md	阅卷标准（硬性通过项 / 质量项 / 典型失分项）	答案与给分标准
env.yaml	环境前提（前置条件 check 命令、关键工件）	考场布置

新出题人不需要理解任何代码，只需要按模板填 4 个文件，5 分钟就能新增一道题。**降低出题门槛 = 题库快速增长 = 评测覆盖度提升。**

这个设计看似简单，背后是几个深思熟虑的决策：

- **分层建设题库。** 基础层（一题一焦点）→ 组合层（能力串联）→ 系统层（端到端）
- **rubric 必须可量化。** 所有"硬性通过项"都必须能从 transcript / 文件系统中找到客观证据，不做含糊判定
- **题面与阅卷分离。** "考生"只能看到 `task.md`，永远看不到`rubric.md` 。避免"对着标准答案抄"

为了同一套题在不同项目复用，题目里所有路径、服务名、需求 ID 统一用占位符，换一个项目接入评测只需要更新"变量配置文件"，所有题目立刻全部生效。

我们把 Harness 工作流的能力拆成了 5 个递进的层次，题库按波次覆盖：

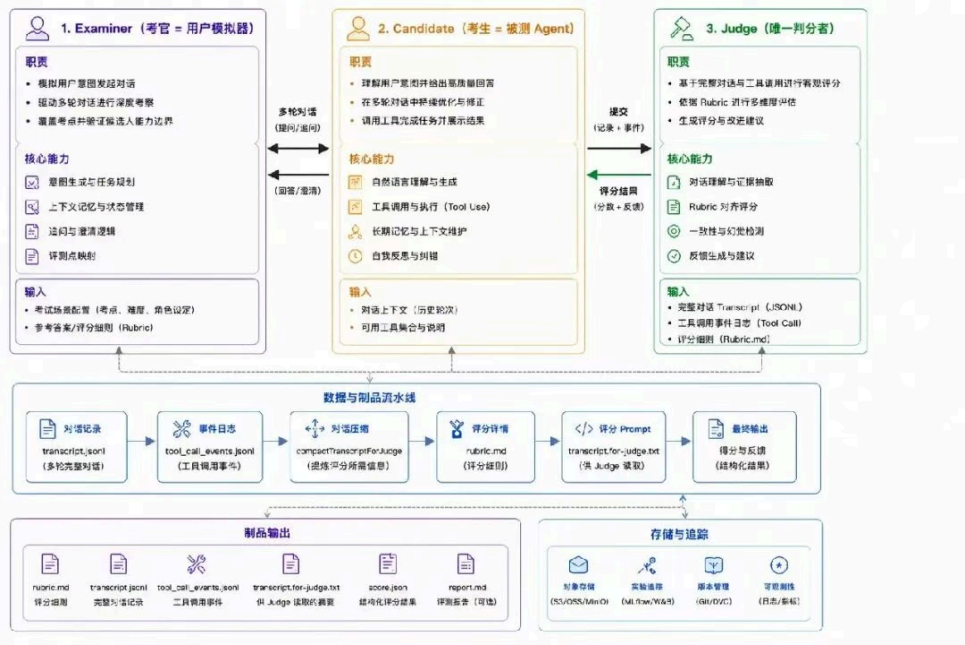
当前题库按“主干闭环→状态机门禁→知识库闭环→周边skill→韧性场景”分5波次铺开：

波次	主题	重点
1	主干闭环	规划 → 执行 → 检查 → 归档 的最小可用闭环
2	状态机 & 门禁	HARD GATE 拦截、分支确认、范围锁定
3	知识库闭环	检索引用 / 写入冲突检测 / 全量审计
4	周边 skill	提交规范、TDD 纪律、发布验证
5	韧性场景	失败修复循环、Critical 拦截、多分支冲突

这个设计背后的思想：先把"主干 path"覆盖住，再依次往"边界场景 / 异常场景"扩展。无论工作流处于哪个发育阶段，都能找到合适难度的题目集合做回归——不至于"题太难全挂"或"题太简单全过"，两种极端都没有信息量。

4.2答题：一场精心编排的“模拟考试”

有题目了，"谁来监考、谁来答题、谁来判分"——



为什么需要考官这个角色?

这是整套系统里最反直觉的设计之一。

最朴素的想法是：直接把 task.md 丢给 Agent，让它跑，跑完看结果。但这完全不符合真实使用场景——真实世界里，用户和 Agent 之间是**多轮交互**的。用户会追问、会纠偏、会补充信息、会在关键节点做决策。

如果评测时去掉了这个交互过程，测的就不是"Agent 在真实场景下的表现"，而是"Agent 自嗨的能力"。**两者差距巨大。**

所以我们引入了**考官（Examiner）**——由 LLM 扮演的"用户"，按照 task.md 中预设的剧本和 Agent 进行多轮对话。

一场完整的"考试"是这样进行的

第一步：考场布置。 系统根据 env.yaml准备好环境——确认所需文件存在、所需服务可达、工作目录正确。相当于考试开始前把试卷、答题卡、草稿纸都摆好。

第二步：考官开场。 考官拿到task.md ，理解"我要扮演的用户角色"和"我要提出的需求"，然后向考生发出第一条消息。这条消息就是用户通常会说的那种话——自然、简洁、可能还带点模糊。

第三步：多轮交互。 考生（被评测的 Agent）收到消息后开始工作。它可能会：

- 提问确认细节（"你说的服务是 A 还是 B？"）
- 调用工具（读文件、跑命令、写代码）
- 汇报进展（"第一步完成了，接下来我要做 X"）
- 请求决策（"有两种方案，你选哪个？"）

考官会根据剧本回应：该确认的确认，该追问的追问，该为难的为难。比如剧本可能规定"如果考生没有主动确认分支名就直接操作，考官要追问一句'你确定是在正确的分支上吗？'"。

第四步：考生完成。 当考生认为任务完成时，会给出最终总结。考官确认对话结束。

第五步：全程录像。 整个对话过程被完整记录为 transcript.jsonl——包括每一轮对话文本、每一次 tool call 的输入输出、每一条 shell 命令的执行结果。这份"录像"是后续判卷的唯一依据。

为什么交互记录如此重要?

这里有一个关键洞察：**Agent 的"能力"不仅体现在最终结果，更体现在过程中的每一步决策。**

两个 Agent 可能都最终完成了同一个任务，但：

- A 在第一步就主动确认了关键参数，B 是被追问后才补上的
- A 在执行前跑了dry-run 验证，B 直接改了线上配置
- A 遇到异常时主动汇报并等待决策，B 自行决定了一个有风险的方案

如果只看最终结果，两者都是"pass"。但过程中的行为质量天差地别。这就是为什么我们需要完整的交互记录——**判卷时不只看"做没做到"，还要看"怎么做到的"。**

为什么判卷独立于对话?

这是踩过的最痛的一个坑。

早期设计里，我们让考官"顺便"判个分。逻辑很自然——考官全程参与了对话，它最了解过程，让它打分不是很合理?

结果发现误判率很高。

原因很简单：考官的视角是对话视角，它只能看到 Agent"说了什么"。但 Agent 的真正工作——写文件、跑命令、调接口——这些都发生在tool call 里，考官在对话层面只看到一句"我已完成操作"。

于是出现了这种荒谬场景：Agent 明明完美执行了所有步骤（从 tool call 记录看），但考官因为在对话里只看到一句简短汇报，判定"考生没有充分展示执行过程"——fail。

> **教训：判分必须拥有完整的"上帝视角"，对话视角远远不够。**

4.3判卷：多维度+证据+改进建议

裁判的阅卷过程

裁判（Judge）是一个完全独立的进程，与答题对话没有任何共享上下文——单独启动、单独运行、没有任何"记忆污染"。它拿到的输入只有两样东西：

- rubric.md：评分标准（只有裁判能看到，考生永远看不到）
- transcript.for-judge.txt：从原始对话记录精简派生的判卷专用版本

我们不是把原始 transcript.jsonl （动辄几万行）原封不动地丢给裁判——那东西太长太噪，裁判会被信息洪水淹没而丢失判断力。我们做了一层专门的压缩处理：**保留所有关键行为证**

据（tool call 内容、文件写入、shell 输出摘要、关键对话节点），去掉冗余信息（重复的系统提示、过长的文件 dump 等）。

这样裁判拥有了完整的"上帝视角"——它能看到考生**真正做了什么**（每一次工具调用、每一行代码写入），而不只是看到考生"嘴上说了什么"。同时信息密度足够高，不会因为噪声而判断失准。

拿到这两份材料后，裁判按以下流程完成阅卷：

1. **逐项核对硬性通过标准。**rubric里每一条"必须达成"的项目，裁判都要transcript中找到对应的行为证据。找不到证据 = 该项不通过。没有"我觉得它应该做了"这种推测空间。
2. **评估过程质量。** 通过了不代表做得好——是主动的还是被动的？是第一时间确认的还是被追问后才补的？是严谨验证过的还是草率跳过的？
3. **给出多维度分数。** 流程遵循度、执行质量、综合分——三个维度各自独立打分，不互相干扰。
4. **引用原文作为证据。** 每个判定都必须附上transcript 中的具体引用片段，做到"有据可查"。不允许出现"考生整体表现不错"这种空泛评语。
5. **强制输出改进建议并分类归因。** 每条建议必须标注维度：[workflow]工作流该补什么规则、[eval]这道题本身有什么问题、[capability] Agent 暴露了什么能力短板。

> 整个过程可以概括为一句话：**拿着标准答案，看着完整录像，逐帧打分，每个判定都要举证。**

裁判输出，分层落库

裁判的输出包含 8 个字段，但我们做了分层落盘设计——核心指标和详细诊断分开存储：

score.yaml（5 字段，结构化指标）：

```yaml
result: pass                          # pass | fail
compliance: 4                         # 流程遵循度 0~5
execution_quality: 4                  # 执行/交付质量 0~5
overall: 4                            # 综合分 0~5
summary: '考生按规范完整跑完五个节点，仅在某步未主动确认参数来源
```

review.md（证据 + 改进建议）：

```yaml
reason: '主动确认服务名 / 参数解析正确 / 写盘前让用户确认 / 命令真实执行'
evidence:
  - '考生 Turn 2: 「请二选一（A 新建 / B 不建）」触发参数验证 skill'
  - '考生 Turn 4: appid: v1_rb4_... → 与 URL 实际值一致'
improvements:
  - '[workflow] Turn 1 未主动确认参数来源，靠用户追问才补上 → 应在对应 skill 增
  - '[eval] rubric 未区分「主动确认」与「被追问后补充」两种达标方式，建议拆分判分项
```

```
8    - '[capability] 多步推理时容易丢失上下文里的关键约束'
```

几个关键设计：

- **pass/fail + 三档分数双轨制。** result 给二元结论，compliance / execution_quality / overall 给细粒度分数（适合趋势图）。一个题里"基本能用但不优雅"和"完美执行"得到的 pass 在分数上被拉开。
- **evidence 必须引用 transcript 原文。** 不允许裁判凭空判定，每条结论都要有可追溯的证据片段。
- **improvements 强制三维度分类：**
  - [workflow]——工作流本身（rules / skills / agents）该补什么 gate、该改什么提示。
  - [eval]——题目本身有没有问题（rubric 模糊、task 误导、env 不合理）。
  - [capability]——考生暴露的通用能力短板（工具使用、上下文理解、多步推理）。

这套分类是反复打磨出来的，目的就是让改进建议可以直接被路由到不同人那里。跑完一批评测 → batch-insights.md 自动汇总 → 直接得到"下一轮工作流要改什么"的清单。**评测不再是"成绩单"，而是工作流的迭代驱动器。**

4.4执行引擎：把各角色串起来，一键执行评测

各个功能角色都已经预备好，怎么把跑题这件事压缩成无人值守的一条命令?

执行引擎是整个系统的"骨架"——用 Go 编写的单一 CLI 程序，编译产物即全部，代码层面只引入了一个外部三方库。没**有任何评测框架、没有任何 ORM、没有任何 web server。**这个选择是为了：零环境部署（go build一下任何机器都能跑）、代码可读性、以及不被框架绑架（评测系统的需求会持续演化，框架越重越难调整）。

执行引擎是整个系统的“骨架”，一条命令背后做的事比想象的多：



几个关键设计：

git worktree 隔离：

每个并发 run 都拿到一份独立的 sandbox 副本——本质上是git worktree 出来的目录树，共享 git 对象库。这一招让"并发跑 5 道题"不会出现 A 题的 `.cursor` 软链覆盖 B 题、A 题的 操作搅进 B 题分支这类问题。

symlink 软链接：

评测期间会用 symlink 把沙箱环境里的 IDE 配置目录（如`.cursor`）替换为本轮 run 内的工作流快照。原始目录（如果有）会先重命名成`.harness-eval.bak` 备份。run 完之后会把用户的真实配置原封不动还回去。相当于考试时把考生桌上的课本收走，换上考卷；考完再把课本还回去。

transient 错误自动重试：

Cursor / Codebuddy 的 CLI 调用难免遇到瞬时失败（网络抖动、CLI 崩溃、error 等）。对话调用（考官/考生每轮交互）：最多 2 次尝试，2 秒间隔，只对 CLI 崩溃和

执行错误重试裁判调用（独立判分）：最多 3 次尝试，3 秒间隔，对所有 transient 错误重试对 `context deadline` 不重试不兜底——超时就是超时，重试只是浪费 token。

4.5评测结果：让成长的趋势“看得见”

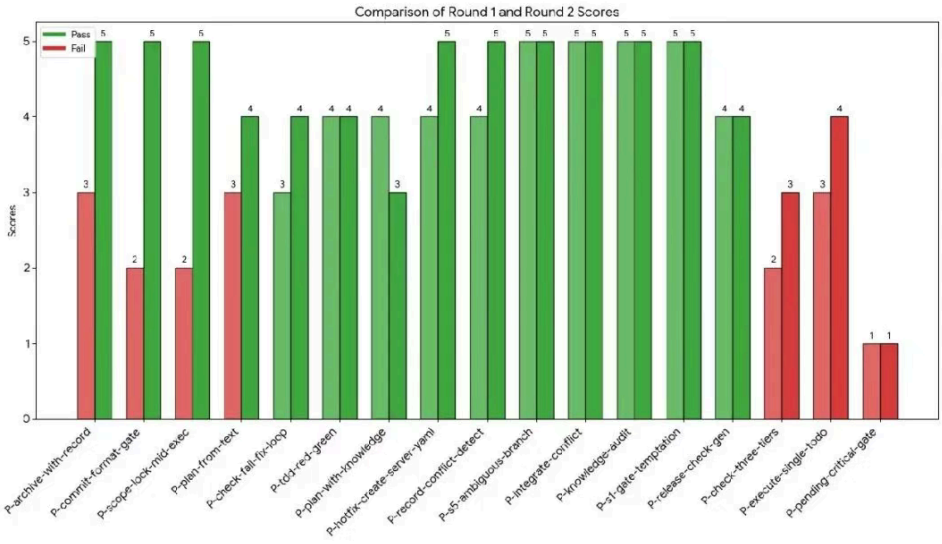
这一节回答："跑了几十次后，怎么从一堆 run 目录里看出工作流到底在变好还是变差。"

产物	内容
`latest.md`	本批 run 的汇总表（每题通过率 / 平均分 / token / 耗时）
`latest-stats.yaml`	同上的结构化版本，便于程序消费
`score-history.yaml`	全历史 run 的扁平记录，每条带 `coworkflow_rev` 字段
`batch-insights.md`	把所有 run 的 improvements 按维度聚类汇总

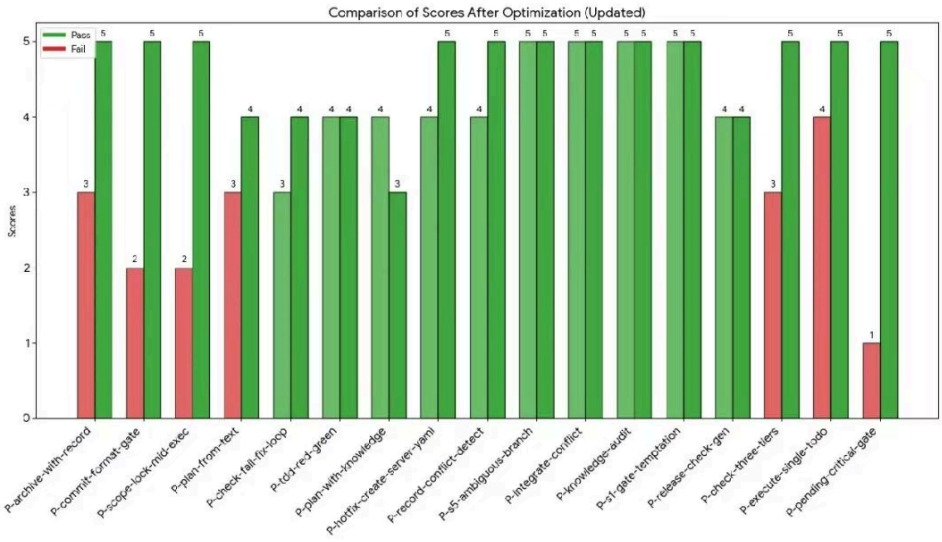
`workflow_rev` 记录每次 run 用的工作流 git commit。这意味着：我们遇到"v0.3 通过率 0.4，v0.4 上提升到 0.8，v0.5 又跌回 0.6——v0.5 改了什么？"这类问题，可以直接定位到某个 commit 的影响，不再靠回忆。

五、评测工程的效果

用这套评测工程在团队内部的 Harness 工作流上进行测试并改进优化，结果如下：



对仍 Fail 的三道题目进行针对性改进后：



整体通过率从之前的 82.4%（14/17）提升到 100%（17/17）。

通过工作流的标准化考题、4 轮 workflow_rev 迭代、50+ 次自动化 run，系统性地暴露了团队内部 Harness 工作流在范围锁定、Critical 拦截、归档沉淀、提交规范、TDD 纪律等多个环节的缺陷，并通过"评测 - 修复 - 回归"闭环逐一修复。多道关键题目的 overall 从 1-2 分修复到满分 5 分。

同时，评测系统本身也沉淀为可复用的工程基础设施和方法论文档，**具备向其他工作流平移的通用性。**

六、写在最后

回过头看，我们做的事情可以抽象为一句话：

> **给一个"行为不确定"的 Harness 工作流，建立可重复的、可归因的、闭环的质量度量体系。**

软件工程界用了几十年总结出"测试驱动开发"。Harness 工作流的复杂度和不确定性比传统软件更高，更需要这套思路：

- 产出**可复现**的评测题目
- 驱动工作流**自动执行**验证
- 根据统一规则**量化评分**
- 基于数据**驱动持续迭代**

这套方法论**不绑定任意平台、不局限任意场景**。但凡存在行为效果难以直观判定优劣的 Harness 类工作流，均可直接使用这套评估体系的思想。

为什么这件事现在特别重要?

Harness 正在从"新鲜玩具"变成"生产工具"。当它真正进入工作流的关键路径时，"好不好用"不再是一个可以靠体感回答的问题——它关系到交付质量、研发效率、甚至线上安全。

任何不可量化的东西都不可优化。 如果不能回答"昨天那版和今天这版谁更好"，所谓的"持续改进"就只是自我感动。

我们只是在团队内部的 Harness 工作流先趟了一遍。如果你也在做类似的事——无论是评测自家的 Harness 工作流、还是想给工作流建立质量基线——欢迎来交流。我们也很想听听不同业务线的评测难点，看看这套评测机制还能做得多通用。

在一个充满概率和不确定性的领域里，"确定性"是最稀缺的资源。而获得确定性的方式，从来都是同一个：量化它，追踪它，用数据说话。





阅读 1.2万

留言

写留言

还没有留言，来写首评